

ПРАКТИЧЕКАЯ РАБОТА

на тему: «Разработка одностраничного web-приложения для персонального учёта фильмов»

по модулю: «Прикладное программирование»

Специальность: 06130100 – Программное обеспечение (по видам)

Квалификация: 4S06130105 – Техник информационных систем

Выполнил: студент 3 курса
8 ТИС/23 группы
Есқали Бейбарыс Жеңісұлы

Допущен к защите «__» _____ 20__ г.
Преподаватель: _____ Турман.Н.Р
(подпись)

Защитил с оценкой:

(буква) (циф.экв.) (балл)

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	2
1 ТЕОРЕТИЧЕСКАЯ ЧАСТЬ: РАЗРАБОТКА WEB-ПРИЛОЖЕНИЯ ДЛЯ УЧЁТА ФИЛЬМОВ	4
1.1 Анализ предметной области web-приложения для учёта фильмов	4
1.2 Проектирование интерфейса web-приложения	5
1.3 Выбор технологий для разработки web-приложения	8
2 ПРАКТИЧЕСКАЯ ЧАСТЬ: СОЗДАНИЕ WEB-ПРИЛОЖЕНИЯ ДЛЯ УЧЁТА ФИЛЬМОВ	10
2.1 Создание проекта и организация структуры папок	10
2.2 Настройка маршрутизации и системы авторизации	11
2.3 Реализация сервисного слоя (storage.js, auth.js)	13
2.4 Реализация бизнес-логики (composables)	15
2.5 Реализация компонентов пользовательского интерфейса	18
2.6 Реализация страниц приложения	21
2.7 Тестирование приложения на соответствие требованиям	24
ЗАКЛЮЧЕНИЕ	27
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	28

ВВЕДЕНИЕ

В современном мире стремительного развития цифровых технологий веб-приложения стали неотъемлемой частью повседневной жизни человека. Одной из актуальных задач является создание удобных персональных инструментов для организации и учёта информации. В частности, любители кино нередко сталкиваются с проблемой отслеживания просмотренных и запланированных к просмотру фильмов: информация хранится разрозненно — в заметках, мессенджерах или просто в памяти. Разработка специализированного web-приложения позволяет решить эту проблему, предоставив пользователю удобный и структурированный инструмент учёта.

Актуальность данной темы обусловлена растущим спросом на лёгкие клиентские приложения, не требующие серверной инфраструктуры и работающие непосредственно в браузере. Современные технологии, в частности фреймворк Vue 3 в сочетании со сборщиком Vite, позволяют создавать полноценные одностраничные приложения (SPA) с реактивным интерфейсом, сохранением данных на стороне клиента и богатым пользовательским опытом.

Объектом исследования данной курсовой работы является процесс разработки клиентского web-приложения на основе современного JavaScript-фреймворка.

Предметом исследования выступают методы и инструменты построения SPA с использованием Vue 3, Composition API, Vue Router и LocalStorage в качестве хранилища данных.

Цель курсовой работы — спроектировать и реализовать одностраничное web-приложение «CineTrack» для персонального учёта фильмов, обеспечивающее полный цикл операций с данными: добавление, редактирование, удаление, фильтрацию и просмотр статистики.

Для достижения поставленной цели необходимо решить следующие задачи:

- провести анализ предметной области и изучить существующие аналогичные решения;
- спроектировать структуру страниц, навигацию и пользовательский интерфейс приложения;
- обосновать выбор технологического стека;
- реализовать компонентную архитектуру приложения на базе Vue 3;
- реализовать сервисный слой для работы с LocalStorage;
- реализовать систему авторизации пользователя на основе LocalStorage;
- провести тестирование готового приложения на соответствие техническим требованиям.

В процессе написания курсовой работы использовались следующие методы исследования: анализ предметной области и существующих решений, метод компонентного проектирования интерфейса, практический метод реализации и тестирования программного продукта.

Курсовая работа состоит из введения, двух глав, заключения, списка использованных источников и приложений. В первой главе рассматриваются теоретические основы разработки: анализ предметной области, проектирование интерфейса и обоснование технологий. Во второй главе описывается практическая реализация приложения по этапам — от создания структуры проекта до тестирования готового продукта.

1 ТЕОРЕТИЧЕСКАЯ ЧАСТЬ: РАЗРАБОТКА WEB-ПРИЛОЖЕНИЯ ДЛЯ УЧЁТА ФИЛЬМОВ

1.1 Анализ предметной области web-приложения для учёта фильмов

Предметная область данного проекта — персональный учёт кинематографического контента. Под учётом фильмов понимается процесс формирования и ведения пользователем собственного списка кинопроизведений с возможностью фиксации статуса просмотра, классификации по жанрам и управления записями на протяжении всего жизненного цикла — от добавления до удаления.

1.1.1. Описание цели и задач проекта

Основной целью проекта является создание удобного клиентского инструмента, позволяющего пользователю вести персональный список фильмов непосредственно в браузере без необходимости регистрации на сторонних сервисах и без зависимости от серверной инфраструктуры. Приложение должно обеспечивать полный CRUD-цикл работы с данными: создание новой записи, просмотр списка, редактирование существующей записи и удаление. Дополнительно приложение предоставляет функции мягкого удаления с возможностью восстановления записи, просмотра статистики и настройки внешнего вида интерфейса. (приложение 1, рисунок 1)

Задачи, решаемые приложением с точки зрения конечного пользователя:

- хранение списка фильмов с описанием и жанровой принадлежностью;
- отметка фильма как просмотренного с визуальным отражением статуса;
- фильтрация списка по жанру, статусу просмотра и поисковому запросу;
- архивирование удалённых записей с возможностью восстановления;
- просмотр статистики по общему количеству фильмов, просмотренным и запланированным;
- персонализация интерфейса: выбор цветовой темы и режима отображения.
- поиск фильмов по названию в глобальной библиотеке TMDb с автоматическим заполнением полей формы;
- отображение детальной страницы фильма с полным описанием, постером и кнопками управления;
- публичная лендинг-страница приложения, доступная без авторизации.

1.1.2. Анализ целевой аудитории

Целевую аудиторию приложения составляют частные пользователи, увлечённые кинематографом и желающие систематизировать свой опыт просмотра. Можно выделить несколько характерных групп пользователей.

Первая группа — активные зрители, регулярно смотрящие фильмы и желающие вести учёт увиденного, чтобы не забыть впечатления и не пересматривать уже просмотренное. Для этой группы критически важны быстрое добавление записей и удобная отметка статуса.

Вторая группа — пользователи с длинными очередями к просмотру, которые накапливают рекомендации от друзей, из интернета и различных источников. Для них приоритетными являются функции поиска, фильтрации и сортировки по жанрам.

Третья группа — технически грамотные пользователи, ценящие приватность и не желающие создавать аккаунты на сторонних сервисах. Их привлекает хранение данных локально, в браузере, без передачи информации на внешние серверы.

Общие требования аудитории: простой и интуитивно понятный интерфейс, быстрый отклик приложения, корректная работа на мобильных устройствах и планшетах.

1.1.3. Обзор аналогичных проектов

На рынке существует ряд сервисов, решающих схожие задачи. Анализ наиболее популярных из них позволяет выявить их сильные и слабые стороны применительно к задачам данного проекта.

Letterboxd (letterboxd.com) — социальная сеть для любителей кино с развитыми функциями учёта и рецензирования. Сильные стороны: большое сообщество, интеграция с базами данных фильмов, развитая социальная составляющая. Слабые стороны: требует обязательной регистрации, данные хранятся на сторонних серверах, приложение перегружено социальными функциями, избыточными для персонального использования.

Кинопоиск (kinopoisk.ru) — крупнейший русскоязычный сервис с обширной базой фильмов и функцией личных списков. Сильные стороны: богатая база данных, интеграция с онлайн-кинотеатрами. Слабые стороны: обязательная авторизация, зависимость от серверной инфраструктуры, избыточный функционал для простого личного учёта.

IMDb Lists (imdb.com) — функция создания списков фильмов на платформе IMDb. Сильные стороны: авторитетная база данных. Слабые стороны: неудобный интерфейс управления списками, требует аккаунта, интерфейс ориентирован прежде всего на поиск, а не на ведение личного трекера.

Сравнительный анализ аналогов показывает общую закономерность: существующие решения либо перегружены функционалом, либо требуют обязательной регистрации и постоянного интернет-соединения. Разрабатываемое приложение CineTrack занимает нишу лёгкого автономного трекера, работающего полностью на стороне клиента.

1.1.4. Определение функциональных требований

На основании анализа предметной области и целевой аудитории были сформулированы следующие функциональные требования к приложению.

Обязательные требования: регистрация и вход пользователя с хранением данных в LocalStorage; добавление фильма с указанием названия, описания и жанра; редактирование существующей записи; мягкое удаление записи с переносом в архив; восстановление записи из архива; безвозвратное удаление из архива; отметка фильма как просмотренного; фильтрация списка по жанру, статусу и поисковому запросу; отображение счётчиков общего количества фильмов, просмотренных и запланированных к просмотру; маршрутизация между страницами без перезагрузки браузера; корректное отображение на экранах от 320px.

Дополнительные требования: страница статистики с прогресс-баром и разбивкой по жанрам; страница настроек с выбором темы оформления, цветовой схемы и режима отображения списка; визуальный индикатор загрузки при переходе между страницами; страница 404 с кнопкой возврата на главную.

1.2 Проектирование интерфейса web-приложения

Проектирование интерфейса является важным этапом разработки, предшествующим написанию кода. На данном этапе определяется структура страниц, логика навигации, визуальная иерархия элементов и общая концепция оформления. Грамотно спроектированный интерфейс снижает количество итераций при разработке и обеспечивает согласованность визуального опыта пользователя.

1.2.1. Разработка структуры страниц и навигации

Приложение CineTrack построено по принципу одностраничного приложения (SPA), в котором переключение между разделами происходит без перезагрузки браузера посредством клиентской маршрутизации. Структура приложения включает восемь маршрутов, каждый из которых соответствует отдельному экрану.

Маршрут `/login` — страница авторизации. Является единственным публично доступным маршрутом. Все остальные маршруты защищены навигационным охранником (router guard) и недоступны без активной пользовательской сессии.

Маршрут `/` — главная страница. Отображает список активных фильмов с фильтрами, счётчиками и элементами управления каждой записью.

Маршрут `/create` — страница создания новой записи. Содержит форму с тремя обязательными полями и кнопками подтверждения и отмены.

Маршрут `/edit/:id` — страница редактирования существующей записи. Форма предзаполняется данными выбранного фильма. При передаче несуществующего идентификатора отображается сообщение об ошибке.

Маршрут `/archive` — страница архива. Отображает мягко удалённые записи с возможностью их восстановления или безвозвратного удаления.

Маршрут `/stats` — страница статистики. Отображает агрегированные данные: общий прогресс просмотра, счётчики по статусам и разбивку по жанрам в виде горизонтальных полос.

Маршрут `/settings` — страница настроек. Позволяет изменить тему оформления, цветовую схему, режим отображения списка и параметр подтверждения удаления.

Маршрут `/about` — страница с описанием проекта, использованного стека технологий и реализованного функционала.

Маршрут `/:pathMatch(*)` — страница 404, отображаемая при обращении к несуществующему адресу.

1.2.2. Описание навигационной системы

Навигация реализована через постоянный заголовок (HeaderNav), присутствующий на всех страницах приложения, кроме страницы авторизации. Заголовок содержит логотип-ссылку на главную страницу, горизонтальное меню с пунктами навигации, кнопку быстрого добавления фильма, отображение имени текущего пользователя и кнопку выхода из системы.

На экранах с шириной менее 768 пикселей горизонтальное меню скрывается, а вместо него отображается кнопка-бургер, раскрывающая вертикальное мобильное меню. Активный пункт меню выделяется визуально посредством изменения цвета и фонового выделения, что обеспечивает пользователю понимание текущего местоположения в приложении.

Описание wireframe-макетов

Ниже описаны схематичные макеты ключевых экранов приложения.

Страница авторизации (/login). В центре экрана располагается карточка фиксированной ширины (максимум 400px). В верхней части карточки — логотип приложения. Под логотипом — переключатель режимов «Войти» и «Регистрация», выполненный в виде сегментированного элемента управления. Ниже располагаются два текстовых поля: «Логин» и «Пароль». Поле пароля содержит кнопку переключения видимости символов. В нижней части карточки — кнопка отправки формы на всю ширину и ссылка для переключения между режимами входа и регистрации.

Главная страница (/). Страница состоит из четырёх зон. Верхняя зона — заголовок страницы с названием раздела, подзаголовком и кнопкой добавления фильма. Вторая зона — строка счётчиков, отображающая общее количество фильмов, количество запланированных к просмотру и просмотренных в виде горизонтального ряда таблеток. Третья зона — панель фильтров: поле поиска, выпадающий список жанров, выпадающий список статусов и кнопка сброса фильтров (отображается только при активных фильтрах). Четвёртая зона — список фильмов, отображаемый в режиме сетки карточек или таблицы в зависимости от настройки пользователя.

Карточка фильма содержит: жанровую метку и метку статуса просмотра в верхней строке; название фильма крупным шрифтом; обрезанное до трёх строк описание; нижнюю строку с датой добавления и тремя кнопками действий — отметить просмотренным, редактировать и удалить.

Страница создания и редактирования (/create, /edit/:id). Страница содержит ссылку возврата назад, заголовок и форму. Форма включает три поля: текстовое поле названия, многострочное поле описания и выпадающий список жанра. Каждое поле снабжено меткой и блоком для вывода ошибки валидации. В нижней части формы расположены кнопки «Отмена» и «Сохранить».

1.2.2.1. Обоснование выбора цветовой схемы и типографики

При проектировании визуального стиля приложения ставилась задача создать интерфейс, сочетающий эстетику кинематографической культуры с современными стандартами UI-дизайна.

В качестве основного акцентного цвета выбран насыщенный терракотово-оранжевый (#c8410a), ассоциирующийся с теплом кинозала, ретро-эстетикой кинопостеров и обложками классических фильмов. Этот цвет применяется для выделения активных элементов, кнопок действий, меток жанров и прогресс-баров. Дополнительно пользователю предоставляется возможность выбора из трёх альтернативных цветовых схем: синей (Indigo), бирюзовой (Teal) и розовой (Rose).

Поддержка двух тем — светлой и тёмной — реализована через CSS-переменные, что позволяет переключать всю палитру приложения без изменения компонентного кода. Все цвета фона, текста, рамок и поверхностей определены через переменные, значения которых переопределяются при установке атрибута data-theme="dark" на корневом элементе документа.

Для типографики выбраны два шрифта. Syne — геометрический гротеск с выраженным характером — используется для заголовков, логотипа и числовых значений счётчиков. Его нестандартные пропорции придают интерфейсу индивидуальность. DM Sans — нейтральный, хорошо читаемый шрифт — используется для основного текста, описаний, меток форм и всех элементов интерфейса, где приоритетом является

разборчивость. Оба шрифта подключаются через Google Fonts и относятся к категории открытых лицензий.

1.3 Выбор технологий для разработки web-приложения

Выбор технологического стека является одним из ключевых архитектурных решений при разработке любого программного продукта. Обоснованный выбор инструментов определяет производительность приложения, удобство его сопровождения и масштабируемость в будущем. В данном разделе рассматриваются технологии, применённые при разработке приложения CineTrack, и обосновывается целесообразность их использования.

1.3.1. Vue 3 + Vite и Composition API

Vue 3 — прогрессивный JavaScript-фреймворк для построения пользовательских интерфейсов. Ключевым нововведением третьей версии стал Composition API — подход к организации логики компонентов, при котором связанный код группируется по функциональному признаку, а не по типу опции. Это принципиально отличает его от Options API, применявшегося во Vue 2, где логика была жёстко разделена на секции data, methods, computed и watch вне зависимости от смысловой связи между свойствами.

Синтаксис script setup является синтаксическим сахаром над Composition API и позволяет писать логику компонента непосредственно в блоке script без необходимости возвращать данные из функции setup. Это сокращает объём шаблонного кода и делает компоненты более лаконичными и читаемыми.

Выбор Vue 3 обусловлен следующими преимуществами. Во-первых, реактивная система на основе Proxy обеспечивает автоматическое отслеживание зависимостей и точечное обновление DOM без дополнительной конфигурации. Во-вторых, однофайловые компоненты (Single File Components, SFC) позволяют хранить шаблон, логику и стили одного компонента в едином файле с расширением .vue, что упрощает навигацию по проекту. В-третьих, Vue 3 имеет значительно меньший размер итогового бандла по сравнению с предыдущей версией за счёт поддержки tree-shaking — механизма исключения неиспользуемого кода на этапе сборки.

Vite — современный инструмент сборки и разработки, разработанный создателем Vue Эваном Ю. В режиме разработки Vite не собирает весь проект целиком, а использует нативные ES-модули браузера, отдавая файлы по требованию. Это обеспечивает практически мгновенный старт сервера разработки вне зависимости от размера проекта. Горячая замена модулей (Hot Module Replacement, HMR) в Vite работает значительно быстрее аналогичных решений, что существенно ускоряет процесс разработки.

1.3.2. Vue Router 4 и навигационные охранники

Vue Router является официальной библиотекой маршрутизации для Vue и в четвёртой версии полностью адаптирован для работы с Vue 3 и Composition API. Библиотека реализует клиентскую маршрутизацию, при которой переключение между страницами происходит без обращения к серверу и без полной перезагрузки браузера, что является фундаментальным принципом одностраничных приложений.

Ключевым механизмом Vue Router, применённым в данном проекте, являются навигационные охранники (navigation guards). Глобальный охранник beforeEach

выполняется перед каждым переходом по маршруту и позволяет реализовать два важных сценария. Первый — защита маршрутов от неавторизованного доступа: если пользователь пытается перейти на любую страницу, кроме /login, при отсутствии активной сессии, охранник автоматически перенаправляет его на страницу авторизации. Второй — обратное перенаправление: если пользователь с активной сессией пытается открыть страницу /login, он автоматически перенаправляется на главную страницу.

Охранник `afterEach` выполняется после завершения каждого перехода и используется для скрытия индикатора загрузки с небольшой задержкой, обеспечивая плавность визуального перехода между страницами.

1.3.3. LocalStorage как сервис хранения данных

`LocalStorage` — встроенный механизм браузера для хранения данных в формате ключ-значение на стороне клиента. В отличие от `SessionStorage`, данные в `LocalStorage` сохраняются между сессиями браузера и не удаляются при закрытии вкладки или окна. Максимальный объём хранилища составляет, как правило, от 5 до 10 мегабайт в зависимости от браузера, что полностью достаточно для хранения персонального списка фильмов.

Для реализации глобальной библиотеки фильмов используется открытый API сервиса The Movie Database (TMDb). TMDb предоставляет бесплатный доступ к базе данных, содержащей миллионы фильмов с описаниями, жанрами, постерами и рейтингами. Обращение к API выполняется через браузерный метод `fetch` без дополнительных библиотек. Для исключения избыточных сетевых запросов при вводе текста реализован механизм `debounce` с задержкой 350 миллисекунд — запрос к серверу выполняется только после того, как пользователь прекратил ввод.

В данном проекте принято архитектурное решение об изоляции всей работы с `LocalStorage` в отдельном сервисном файле `services/storage.js`. Это означает, что ни один компонент и ни один `composable` не обращается к `LocalStorage` напрямую — все операции чтения и записи осуществляются исключительно через методы сервисного слоя. Данный подход обеспечивает несколько важных преимуществ.

Во-первых, централизованная обработка ошибок: все вызовы `JSON.parse` и `JSON.stringify` обернуты в блоки `try/catch`, что предотвращает аварийное завершение работы приложения при повреждённых или отсутствующих данных. Во-вторых, единое место хранения ключей: все строки-идентификаторы записей `LocalStorage` вынесены в константы в начале файла, что исключает возможность опечаток при обращении к одному и тому же ключу из разных мест кода. В-третьих, заменяемость хранилища: при необходимости перехода на другой механизм хранения данных, например `IndexedDB` или серверный API, достаточно изменить только сервисный файл, не затрагивая остальную кодовую базу.

1.3.4. Подход `singleton composables`

`Composables` — это функции, инкапсулирующие реактивное состояние и связанную с ним логику с использованием `Composition API`. В отличие от компонентного состояния, которое создаётся заново при каждом монтировании компонента, `singleton composable` хранит своё состояние на уровне модуля JavaScript. Это означает, что реактивные переменные объявляются вне тела экспортируемой функции, и при повторном вызове функции возвращаются ссылки на уже существующие объекты, а не создаются новые.

В приложении CineTrack по данному принципу реализованы четыре composable. useItems хранит единый реактивный массив фильмов и предоставляет все методы работы с ним. useSettings хранит настройки пользователя и синхронизирует их с LocalStorage при каждом изменении.

2 ПРАКТИЧЕСКАЯ ЧАСТЬ: СОЗДАНИЕ WEB-ПРИЛОЖЕНИЯ ДЛЯ УЧЁТА ФИЛЬМОВ

2.1 Создание проекта и организация структуры папок

Первым этапом практической реализации является создание проекта и формирование его файловой структуры. Правильная организация файлов с самого начала разработки обеспечивает читаемость кода, удобство навигации по проекту и соответствие принятым в профессиональном сообществе стандартам.

Создание проекта

Для создания проекта используется официальный инструмент Vite. В терминале выполняется следующая команда:

```
npm create vite@latest cinetrack -- --template vue
```

После выполнения команды Vite генерирует базовую структуру проекта с предустановленными зависимостями для Vue 3. Затем выполняется переход в папку проекта и установка зависимостей:

```
cd cinetrack
```

```
npm install
```

Дополнительно устанавливается библиотека Vue Router, не входящая в базовый шаблон Vite:

```
npm install vue-router@4
```

После завершения установки запускается сервер разработки командой `npm run dev`. Vite запускает локальный сервер по адресу `http://localhost:5173`, и приложение становится доступным в браузере.

Организация структуры папок

Структура папок проекта формируется в соответствии с принципом разделения ответственности. Каждая папка содержит файлы строго определённого типа и назначения, что позволяет любому разработчику быстро ориентироваться в проекте.

Папка `components` содержит переиспользуемые компоненты пользовательского интерфейса, которые не привязаны к конкретному маршруту и могут использоваться на нескольких страницах. К таким компонентам относятся элементы постоянного макета — шапка, подвал и индикатор загрузки — а также компоненты, реализующие повторяющуюся логику отображения: карточка фильма, список фильмов, форма добавления и редактирования, панель фильтров.

Папка `views` содержит компоненты-страницы, каждый из которых соответствует отдельному маршруту приложения. Компоненты-страницы не содержат самостоятельной логики хранения данных, а получают данные и методы из `composables`, komponуя их с компонентами из папки `components`.

Папка `composables` содержит функции, реализующие реактивное состояние и бизнес-логику приложения. Все `composables` реализованы по принципу `singleton`: их состояние хранится на уровне модуля и является единым для всего приложения.

Папка `services` содержит модули, отвечающие за взаимодействие с внешними источниками данных. В данном проекте таким источником является `LocalStorage` браузера.

Сервисный слой полностью изолирует остальную кодовую базу от прямого обращения к браузерному API.

Папка router содержит единственный файл index.js с конфигурацией маршрутов и навигационных охранников.

Точкой входа в приложение является файл main.js, в котором создаётся экземпляр Vue-приложения, подключается роутер и выполняется монтирование в DOM-элемент с идентификатором app. Файл App.vue является корневым компонентом и определяет общий макет приложения, включающий постоянные элементы — шапку, индикатор загрузки и подвал — а также точку монтирования страниц в виде компонента router-view.

Данный файл намеренно сохраняется минимальным: подключение глобальных стилей, инициализация данных и настройка темы выполняются в соответствующих модулях — App.vue и composables — а не в точке входа. Это соответствует принципу единственной ответственности и упрощает тестирование отдельных модулей.

2.2 Настройка маршрутизации и системы авторизации

Маршрутизация и авторизация реализуются совместно, поскольку навигационный охранник роутера является точкой применения логики защиты маршрутов. На данном этапе создаются файл router/index.js, сервис services/auth.js и composable useAuth.js.

Файл router/index.js создаётся в папке router и является единственным местом в проекте, где определяется вся маршрутная карта приложения. Роутер создаётся функцией createRouter с использованием режима истории createWebHistory, при котором URL-адреса имеют стандартный вид без символа решётки.

Каждый маршрут описывается объектом с двумя обязательными полями: path определяет адрес маршрута, component указывает компонент-страницу, который будет отображаться по данному адресу. Дополнительное поле meta используется для передачи произвольных метаданных маршрута. В данном проекте поле meta применяется для маркировки публичных маршрутов: маршрут /login получает флаг meta: { public: true }, что позволяет навигационному охраннику определить, требует ли данная страница авторизации.

Полный список маршрутов приложения приведён в таблице 1.

Таблица 1 — Маршруты приложения CineTrack

№	Путь	Компонент	Доступ	Описание
1	2	3	4	5
1	/login	LoginView	Публичный	Страница авторизации и регистрации
2	/	LandingView	Защищённый	Главная страница со приветствием
3	/create	CreateView	Защищённый	Форма добавления нового фильма

Продолжение таблицы 1

1	2	3	4	5
4	/edit/:id	EditView	Защищённый	Форма редактирования фильма по идентификатору
5	/archive	ArchiveView	Защищённый	Архив удалённых фильмов
6	/stats	StatsView	Защищённый	Страница статистики
7	/settings	SettingsView	Защищённый	Страница настроек
8	/about	AboutView	Защищённый	Информация о проекте
9	/:pathMatch(.*)	NotFoundView	Защищённый	Страница 404
10	/films	HomeView	Защищённый	Главная страница со списком фильмов
11	/film:id	FilmView	Защищённый	Страница с подробной информацией о фильме
Примечание - составлено автором на основе проведенного исследования				

2.2.2. Навигационные охранники

Охранник `beforeEach` выполняется перед каждым переходом по маршруту и решает две задачи, как показано на рисунке 2 в приложениях 2. Первая — активация индикатора загрузки через вызов функции `showLoader` из `composable useLoader`. Вторая — проверка авторизации пользователя. Охранник получает текущую сессию из сервиса `auth.js` и проверяет два условия: если маршрут не помечен как публичный и сессия отсутствует, выполняется перенаправление на `/login`; если маршрут равен `/login` и сессия активна, выполняется перенаправление на главную страницу.

Охранник `afterEach` выполняется после завершения каждого перехода. Его единственная задача — скрыть индикатор загрузки через вызов `hideLoader` с задержкой 400 миллисекунд. Задержка обеспечивает минимально заметную анимацию перехода даже при мгновенной смене маршрута, что улучшает ощущение отклика интерфейса.

2.2.3. Система авторизации

Авторизация реализована полностью на стороне клиента с использованием `LocalStorage`. Архитектурно система состоит из трёх взаимосвязанных уровней: сервис `auth.js`, `composable useAuth.js` и компонент-страница `LoginView.vue`.

Сервис `auth.js` хранит всю логику работы с данными пользователей и сессией. Пользователи хранятся в `LocalStorage` в виде массива объектов под ключом `cinetrack_users`. Каждый объект пользователя содержит уникальный идентификатор, логин, хэш пароля и дату регистрации. Активная сессия хранится отдельно под ключом `cinetrack_session` в виде объекта с идентификатором и логином пользователя.

Для исключения хранения паролей в открытом виде реализована функция хэширования на основе простого алгоритма: каждый символ пароля влияет на

результатирующее числовое значение через побитовые операции, итоговое число преобразуется в шестнадцатеричную строку. Данный подход не является криптографически стойким и применяется исключительно в учебных целях для демонстрации принципа хранения хэшей вместо открытых паролей.

Сервис предоставляет четыре функции. Функция `registerUser` принимает логин и пароль, проверяет уникальность логина, создаёт объект пользователя, сохраняет его в массив и немедленно создаёт сессию. Функция `loginUser` принимает логин и пароль, находит пользователя в хранилище по совпадению логина и хэша пароля, при успехе создаёт сессию. Функция `getSession` возвращает текущую сессию из `LocalStorage` или `null` при её отсутствии. Функция `logoutUser` удаляет запись сессии из `LocalStorage`.

`Composable useAuth.js` оборачивает функции сервиса в реактивный слой. Текущий пользователь хранится в реактивной переменной `currentUser`, инициализируемой при загрузке модуля через вызов `getSession`. Это обеспечивает автоматическое восстановление состояния авторизации при перезагрузке страницы. Из `composable` экспортируются вычисляемые свойства `isLoggedIn` и `userName`, используемые компонентами для условного отображения элементов интерфейса.

Компонент `LoginView.vue` реализует единую страницу с двумя режимами: вход и регистрация. Переключение между режимами осуществляется без перехода на другой маршрут — изменяется только локальная переменная `mode`, что меняет заголовок формы, текст кнопки отправки и подсказку в нижней части карточки. Форма содержит валидацию обоих полей: логин должен содержать не менее трёх символов, пароль — не менее четырёх. При ошибке, возвращённой сервисом — например, при попытке зарегистрироваться с уже существующим логином или при вводе неверного пароля — сообщение отображается в отдельном блоке над кнопкой отправки. При успешной авторизации или регистрации роутер выполняет перенаправление на главную страницу.

2.3 Реализация сервисного слоя (`storage.js`, `auth.js`)

Сервисный слой является фундаментом архитектуры приложения `CineTrack`. Его назначение — полная изоляция всей логики работы с `LocalStorage` от остальной кодовой базы. Ни один компонент, ни одна страница и ни один `composable` не обращаются к объекту `localStorage` напрямую. Все операции чтения и записи данных осуществляются исключительно через методы двух сервисных модулей: `storage.js` и `auth.js`.

Данное архитектурное решение обеспечивает три ключевых преимущества. Во-первых, централизованная обработка ошибок позволяет перехватывать исключения в одном месте, не допуская аварийного завершения работы приложения при повреждённых данных. Во-вторых, единое хранение ключей исключает возможность опечаток при обращении к одному и тому же ключу из разных частей кода. В-третьих, заменяемость хранилища означает, что при необходимости перехода на другой механизм хранения достаточно изменить только сервисный файл.

Реализация модуля `storage.js`

Файл `services/storage.js` отвечает за хранение и загрузку двух типов данных: списка фильмов и пользовательских настроек.

Вынесение ключей в константы является обязательным требованием, зафиксированным в техническом задании. Это исключает ситуацию, при которой одна и та же строка-ключ записывается по-разному в разных местах кода, что привело бы к потере данных.

Модуль предоставляет пять функций. Рассмотрим каждую из них подробно.

Функция `getItems` читает массив фильмов из `LocalStorage`. Она обернута в блок `try/catch`: при успешном чтении строка разбирается функцией `JSON.parse` и возвращается как массив объектов; при возникновении исключения — например, если данные в хранилище повреждены и не являются валидным JSON — в консоль выводится сообщение об ошибке и возвращается пустой массив. Это предотвращает аварийное завершение работы приложения в случае некорректных данных, как показано на рисунке 3.

```
// Чтение фильмов из localStorage
export function getItems() {
  try {
    const raw = localStorage.getItem(KEYS.ITEMS)
    return raw ? JSON.parse(raw) : []
  } catch (e) {
    console.error('[storage] getItems failed:', e)
    return []
  }
}
```

Рисунок 3 — Функция `getItems` (services/[storage.js](#))

Примечание - составлено автором на основе проведенного исследования

Функция `saveItems` принимает массив фильмов, сериализует его в строку JSON и записывает в `LocalStorage`. Также обернута в `try/catch` для обработки возможной ошибки превышения квоты хранилища, как показано на рисунке 4.

```
// Сохранение фильмов в localStorage
export function saveItems(items) {
  try {
    localStorage.setItem(KEYS.ITEMS, JSON.stringify(items))
  } catch (e) {
    console.error('[storage] saveItems failed:', e)
  }
}
```

Рисунок 4 — Функция `saveItems` (services/[storage.js](#))

Примечание - составлено автором на основе проведенного исследования

Функции `getSettings` и `saveSettings` работают по аналогичному принципу применительно к объекту настроек пользователя. `getSettings` возвращает объект настроек или `null` при его отсутствии, что позволяет вызывающему коду применить значения по умолчанию, как показано на рисунке 5.

```
// Чтение настроек из localStorage
export function getSettings() {
  try {
    const raw = localStorage.getItem(KEYS.SETTINGS)
    return raw ? JSON.parse(raw) : null
  } catch (e) {
    console.error('[storage] getSettings failed:', e)
    return null
  }
}
```

Рисунок 5 — Функция `getSettings` (services/[storage.js](#))

Примечание - составлено автором на основе проведенного исследования

Функция `initDataIfEmpty` проверяет, содержит ли хранилище записи фильмов. Если массив пуст, функция записывает в хранилище предустановленный набор демонстрационных данных и возвращает его. Если данные уже существуют, функция возвращает их без изменений, как показано на рисунке 6.

```
// Инициализация начальными данными, если хранилище пусто
export function initDataIfEmpty() {
  const existing = getItems()
  if (existing.length === 0) {
    saveItems(DEMO_ITEMS)
    return DEMO_ITEMS
  }
  return existing
}
```

Рисунок 6 — Функция `initDataIfEmpty` (services/[storage.js](#))

Примечание - составлено автором на основе проведенного исследования

Демонстрационные данные объявлены в константе `DEMO_ITEMS` в том же файле и включают четыре записи фильмов из разных жанров с различными статусами просмотра, что позволяет сразу продемонстрировать все возможности интерфейса.

Файл `services/auth.js` отвечает за регистрацию, аутентификацию и управление сессией пользователя.

Под ключом `cinetrack_users` хранится массив всех зарегистрированных пользователей. Под ключом `cinetrack_session` хранится объект текущей активной сессии, содержащий идентификатор и логин пользователя.

Для исключения хранения паролей в открытом виде реализована вспомогательная функция `hashPassword`. Функция обходит каждый символ строки пароля, применяет к аккумулятору побитовые операции сдвига и сложения, после чего преобразует результат в шестнадцатеричную строку. Важно понимать, что данный алгоритм применяется исключительно в учебных целях и не является криптографически стойким.

Функция `registerUser` принимает логин и пароль. Первым шагом выполняется проверка уникальности логина: если пользователь с таким логином уже существует в массиве, функция возвращает объект с полем `ok: false` и текстом ошибки. При уникальном логине создаётся новый объект пользователя с уникальным идентификатором, сгенерированным через `crypto.randomUUID`, хэшированным паролем и временной меткой регистрации. Объект добавляется в массив пользователей, массив сохраняется в `LocalStorage`, после чего немедленно создаётся сессия и возвращается объект с полем `ok: true`.

Функция `loginUser` принимает логин и пароль. Она выполняет поиск пользователя в массиве по совпадению двух условий одновременно: логин должен совпадать точно, а хэш введённого пароля должен совпадать с хэшем, хранящимся в записи пользователя. Если совпадение найдено, создаётся сессия и возвращается объект успеха. Если нет — возвращается объект с сообщением об ошибке. Намеренно используется единое сообщение «Неверный логин или пароль» без уточнения, какое именно поле содержит ошибку, что является стандартной практикой безопасности, не позволяющей определить существование аккаунта по реакции формы.

Функция `getSession` читает объект сессии из `LocalStorage` и возвращает его или `null`. Она используется в двух местах: при инициализации `composable useAuth` для восстановления состояния авторизации после перезагрузки страницы, и в навигационном охраннике роутера для проверки наличия активной сессии перед каждым переходом.

Функция `logoutUser` удаляет запись сессии из `LocalStorage` вызовом `localStorage.removeItem`. Данные пользователя и список фильмов при выходе не удаляются, что позволяет пользователю войти повторно и обнаружить все свои данные в сохранённом виде.

Взаимодействие сервисного слоя с остальными модулями

Сервисные модули намеренно не содержат реактивного состояния и не импортируют ничего из `Vue`. Они являются чистыми JavaScript-модулями, работающими с браузерным API. Это делает их полностью независимыми от фреймворка и упрощает возможное тестирование. Реактивная обёртка над сервисами реализована на уровне `composables`, которые рассматриваются в следующем разделе.

Схема взаимодействия слоёв приложения с точки зрения потока данных выглядит следующим образом: компоненты и страницы вызывают методы `composables`; `composables` вызывают функции сервисного слоя при каждом изменении состояния; сервисный слой выполняет чтение и запись в `LocalStorage`. Обратный поток: при инициализации приложения сервисный слой читает данные из `LocalStorage`, `composables` получают эти данные и помещают их в реактивные переменные, компоненты автоматически отрисовываются на основе реактивного состояния.

2.4 Реализация бизнес-логики (composables)

Composables представляют собой центральный слой архитектуры приложения CineTrack, расположенный между сервисным слоем и пользовательским интерфейсом. Именно в composables сосредоточена вся бизнес-логика приложения: управление списком фильмов, хранение настроек, контроль состояния загрузки и управление авторизацией. В данном разделе рассматриваются четыре composable, реализованных в папке composables.

2.4.1. Принцип singleton и его реализация

Все четыре composable реализованы по принципу singleton, при котором реактивное состояние объявляется на уровне модуля — вне тела экспортируемой функции. Это принципиальное архитектурное решение, определяющее поведение всего приложения.

Для понимания разницы рассмотрим два подхода. При обычном подходе реактивная переменная объявляется внутри функции composable, и при каждом вызове этой функции создаётся новый независимый экземпляр состояния. При singleton-подходе переменная объявляется вне функции, на уровне модуля, и функция лишь возвращает ссылку на уже существующий объект. Поскольку модули JavaScript кэшируются после первой загрузки, состояние существует в единственном экземпляре на протяжении всей жизни приложения. Любой компонент, вызывающий composable, получает доступ к одному и тому же реактивному объекту, что автоматически обеспечивает согласованность данных без дополнительных механизмов синхронизации.

2.4.2. Реализация useLoader.js

Composable useLoader является простейшим из четырёх и отвечает за управление состоянием глобального индикатора загрузки. Его состояние представлено единственной реактивной переменной isLoading булевого типа, инициализированной значением false.

Composable предоставляет два метода: showLoader устанавливает isLoading в значение true, hideLoader — в значение false. Простота реализации обусловлена узкой ответственностью модуля: он не выполняет никаких операций с данными, а лишь хранит флаг и предоставляет методы его изменения.

Переменная isLoading используется в двух местах приложения: в роутере, где охранные beforeEach и afterEach вызывают showLoader и hideLoader соответственно, и в компоненте AppLoader, который подписывается на isLoading и отображает или скрывает оверлей в зависимости от его значения. Поскольку оба места используют один и тот же singleton, изменение состояния в роутере немедленно отражается в компоненте.

2.4.3. Реализация useSettings.js

Composable useSettings отвечает за хранение пользовательских настроек и их синхронизацию с LocalStorage. Состояние представлено четырьмя реактивными переменными: theme хранит текущую тему оформления и может принимать значения light или dark; colorScheme хранит выбранную цветовую схему из четырёх доступных вариантов; viewMode определяет режим отображения списка фильмов и принимает значения cards или table; confirmDelete хранит булевый флаг, определяющий необходимость показа диалога подтверждения перед удалением записи.

При инициализации модуля выполняется попытка загрузить сохранённые настройки из LocalStorage через вызов getSettings. Если настройки найдены, реактивные переменные

инициализируются их значениями; если нет — используются значения по умолчанию. Сразу после инициализации вызывается вспомогательная функция `applyToDocument`, которая устанавливает атрибуты `data-theme` и `data-scheme` на корневом элементе документа. Это обеспечивает применение сохранённой темы немедленно при загрузке страницы, без мерцания интерфейса.

Каждый из четырёх параметров имеет собственный метод-сеттер: `setTheme`, `setColorScheme`, `setViewMode` и `setConfirmDelete`. Каждый сеттер обновляет реактивную переменную, при необходимости вызывает `applyToDocument` для применения изменений к документу и вызывает функцию `persist` для сохранения актуального состояния настроек в `LocalStorage`. Такая схема гарантирует, что `LocalStorage` всегда содержит актуальное состояние настроек после любого изменения.

2.4.4. Реализация `useAuth.js`

`Composable useAuth` обёртывает функции сервиса `auth.js` в реактивный слой. Состояние представлено единственной реактивной переменной `currentUser`, инициализируемой при загрузке модуля через вызов `getSession`. Если сессия активна, переменная содержит объект с идентификатором и логином пользователя; если нет — значение `null`.

Из `composable` экспортируются два вычисляемых свойства. `isLoggedIn` возвращает булево значение, определяемое наличием объекта в `currentUser`, и используется роутером и компонентами для условного отображения элементов. `userName` возвращает строку логина текущего пользователя и отображается в шапке приложения.

Метод `login` вызывает функцию `loginUser` сервиса и при успешном результате обновляет реактивную переменную `currentUser`. Метод `register` аналогично вызывает `registerUser`. Метод `logout` вызывает `logoutUser` сервиса, устанавливает `currentUser` в `null` и выполняет программный переход на страницу авторизации через переданный экземпляр роутера. Передача роутера в метод `logout` вместо его прямого импорта является намеренным решением, позволяющим избежать циклических зависимостей между модулями.

2.4.5. Реализация `useItems.js`

`Composable useItems` является ядром бизнес-логики приложения и содержит наибольший объём функциональности. Его состояние представлено единственной реактивной переменной `items` — массивом всех записей фильмов, включая как активные, так и мягко удалённые.

При инициализации модуля вызывается функция `initDataIfEmpty` сервиса `storage.js`, которая либо возвращает существующие данные из `LocalStorage`, либо записывает и возвращает демонстрационный набор данных. Результат немедленно присваивается реактивному массиву `items`, что обеспечивает наполнение интерфейса данными при первой же отрисовке.

На основе массива `items` определены четыре вычисляемых свойства. `activeItems` возвращает отфильтрованный массив записей, у которых поле `deletedAt` равно `null`, то есть не удалённых. `archivedItems` возвращает записи с ненулевым значением `deletedAt`. `totalCount` возвращает количество активных записей. `activeCount` возвращает количество активных записей со статусом `isDone` равным `false`. `doneCount` возвращает количество просмотренных активных записей. `deletedCount` возвращает количество архивных записей. Все перечисленные свойства обновляются автоматически при любом изменении массива

items, что обеспечивает реактивное обновление счётчиков и списков на всех страницах приложения одновременно.

Composable предоставляет семь методов управления данными. Рассмотрим каждый из них.

Метод addItem принимает объект с полями title, description и category. Он создаёт новый объект записи, добавляя к переданным данным уникальный идентификатор через crypto.randomUUID, временную метку создания через Date.now, начальное значение isDone равное false и начальное значение deletedAt равное null. Новый объект добавляется в массив items, после чего немедленно вызывается saveItems для синхронизации с LocalStorage.

Метод updateItem принимает идентификатор записи и объект с обновлёнными полями. Он находит индекс записи в массиве и заменяет её слиянием старого и нового объектов через оператор spread, что обеспечивает обновление только переданных полей с сохранением остальных. После обновления вызывается saveItems.

Метод toggleDone находит запись по идентификатору и инвертирует значение её поля isDone. Используется для отметки фильма просмотренным и снятия отметки.

Метод softDelete реализует мягкое удаление: вместо физического удаления записи из массива он устанавливает в её поле deletedAt текущую временную метку. Запись исчезает из activeItems и появляется в archivedItems, оставаясь при этом в массиве items.

Метод restoreItem выполняет обратную операцию: устанавливает deletedAt в null, возвращая запись из архива в активный список.

Метод deleteForever выполняет физическое удаление записи из массива через фильтрацию по идентификатору. Данная операция необратима.

Метод getItemById выполняет поиск записи в полном массиве items по идентификатору и возвращает найденный объект или null. Используется страницей редактирования для предзаполнения формы данными выбранного фильма.

Важной особенностью реализации является то, что saveItems вызывается в конце каждого метода, изменяющего данные. Это гарантирует, что LocalStorage всегда содержит актуальное состояние списка фильмов вне зависимости от того, какая операция была выполнена последней, и данные не будут потеряны при перезагрузке страницы.

2.5 Реализация компонентов пользовательского интерфейса

Компоненты пользовательского интерфейса расположены в папке components и представляют собой переиспользуемые строительные блоки приложения. В отличие от страниц, компоненты не привязаны к конкретному маршруту и могут включаться в состав нескольких страниц одновременно. В приложении CineTrack реализовано семь компонентов, которые можно разделить на две группы: компоненты макета и компоненты предметной области.

2.5.1. Компоненты макета

К компонентам макета относятся HeaderNav, AppFooter и AppLoader. Эти компоненты подключаются непосредственно в корневом файле App.vue и присутствуют на всех страницах приложения независимо от текущего маршрута.

Компонент `HeaderNav` реализует постоянную шапку приложения. Шапка зафиксирована в верхней части экрана через CSS-свойство `position: sticky` и перекрывает остальное содержимое страницы за счёт значения `z-index: 100`. Компонент содержит четыре функциональных блока: логотип-ссылку на главную страницу, горизонтальное навигационное меню для десктопных экранов, блок пользователя с отображением логина и кнопкой выхода, а также кнопку-бургер для мобильных экранов.

Навигационные ссылки реализованы через компонент `router-link` библиотеки `Vue Router`. Активный пункт меню определяется автоматически: `Vue Router` добавляет CSS-класс `router-link-exact-active` к ссылке, чей путь точно совпадает с текущим маршрутом, что позволяет визуально выделить текущий раздел без дополнительной логики в компоненте.

Мобильное меню управляется локальной реактивной переменной `menuOpen`. При нажатии на кнопку-бургер переменная инвертируется, что показывает или скрывает вертикальное меню. Появление мобильного меню сопровождается анимацией плавного раскрытия, реализованной через компонент `transition` `Vue` с CSS-переходом по свойствам `max-height` и `opacity`. При выборе любого пункта мобильного меню вызывается метод `closeMenu`, закрывающий меню до совершения навигационного перехода.

Для выхода из системы компонент вызывает метод `logout` из `composable useAuth`, передавая текущий экземпляр роутера. После очистки сессии роутер выполняет перенаправление на страницу авторизации.

Компонент `AppFooter` реализует постоянный подвал приложения. Компонент содержит название и слоган приложения, а также текущий год, вычисляемый динамически через объект `Date` `JavaScript`. Это исключает необходимость ручного обновления года в коде.

Компонент `AppLoader` реализует полноэкранный оверлей индикатора загрузки. Компонент подписывается на реактивную переменную `isLoading` из `composable useLoader` и отображается или скрывается в зависимости от её значения. Появление и исчезновение оверлея сопровождаются анимацией изменения прозрачности через компонент `transition`. Оверлей перекрывает всё содержимое страницы за счёт фиксированного позиционирования и значения `z-index: 9999`, а CSS-свойство `pointer-events: all` блокирует все взаимодействия пользователя с интерфейсом в процессе перехода между страницами. Визуальным индикатором загрузки служит анимированное вращающееся кольцо, реализованное на чистом CSS.

2.5.2. Компоненты предметной области

К компонентам предметной области относятся `ItemCard`, `ItemList`, `ItemForm` и `ItemFilters`. Эти компоненты реализуют логику отображения и взаимодействия непосредственно с данными о фильмах.

Компонент `ItemCard` отображает информацию об одном фильме в виде карточки. Компонент принимает единственный входной параметр `item` — объект записи фильма — и генерирует два пользовательских события: `toggle` при нажатии кнопки отметки просмотра и `delete` при нажатии кнопки удаления. Родительский компонент подписывается на эти события и вызывает соответствующие методы `composable useItems`.

Карточка состоит из четырёх визуальных зон. Верхняя строка содержит жанровую метку с фоновым выделением цветом акцента и метку статуса просмотра, отображаемую только для просмотренных фильмов. Заголовок отображает название фильма крупным

шрифтом семейства Syne. Описание выводится с ограничением в три строки через CSS-свойство `-webkit-line-clamp`, что обеспечивает единообразную высоту карточек в сетке независимо от длины текста. Нижняя строка содержит дату добавления, отформатированную через метод `toLocaleDateString` с русской локалью, и три кнопки действий.

Карточки просмотренных фильмов получают модифицирующий CSS-класс `card--done`, который изменяет фоновый цвет карточки на оттенок зелёного, снижает её непрозрачность до 85% и применяет зачёркивание к заголовку. Это обеспечивает немедленную визуальную обратную связь при отметке фильма просмотренным без необходимости перехода на другую страницу.

Компонент `ItemList` является компонентом-переключателем, отвечающим за выбор режима отображения списка фильмов. Он принимает массив записей и проверяет текущее значение `viewMode` из `composable useSettings`. При значении `cards` компонент отрисовывает сетку из компонентов `ItemCard` через директиву `v-for`. При значении `table` компонент отрисовывает таблицу с пятью столбцами: название, жанр, статус, дата добавления и действия. Оба варианта отображения поддерживают одинаковые операции с записями через одинаковые пользовательские события, что делает переключение режимов прозрачным для родительской страницы.

Компонент `ItemForm` является переиспользуемым компонентом формы, применяемым как на странице создания, так и на странице редактирования фильма. Компонент принимает два входных параметра: `initial` — объект с начальными значениями полей формы, используемый при редактировании существующей записи — и `submitLabel` — строка с текстом кнопки отправки, позволяющая настроить форму под конкретный сценарий использования.

Форма содержит три поля ввода. Текстовое поле `title` принимает название фильма. Многострочное поле `textarea` принимает описание фильма. Выпадающий список `select` позволяет выбрать жанр из одиннадцати предустановленных вариантов: Боевик, Драма, Комедия, Триллер, Ужасы, Фантастика, Аниме, Документальный, Мультфильм, Криминал и Романтика.

Локальная валидация реализована в методе `validate`, вызываемом при отправке формы. Для поля `title` проверяется непустое значение и минимальная длина в два символа. Для поля `description` проверяется непустое значение и минимальная длина в десять символов. Для поля `category` проверяется выбор значения из списка. При обнаружении ошибок валидации соответствующие сообщения сохраняются в реактивном объекте `errors` и отображаются под каждым полем, а поле визуально выделяется красной рамкой. Отправка формы выполняется только при успешном прохождении всех проверок, после чего генерируется пользовательское событие `submit` с объектом данных формы.

Синхронизация формы с входным параметром `initial` реализована через наблюдатель `watch`. Это необходимо для корректной работы страницы редактирования: поскольку `composable useItems` инициализируется до монтирования компонента формы, данные фильма могут поступить в компонент не в момент его создания, а позднее, после завершения навигационного перехода. Наблюдатель отслеживает изменения входного параметра `initial` и при их обнаружении обновляет локальные переменные формы.

Компонент `ItemFilters` реализует панель фильтрации списка фильмов. Компонент использует паттерн `v-model` для двустороннего связывания с объектом фильтров

родительской страницы. Входной параметр `modelValue` содержит текущий объект фильтров с тремя полями: `search`, `category` и `status`. При изменении любого поля компонент генерирует событие `update:modelValue` с обновлённым объектом фильтров, не мутируя входной параметр напрямую, что соответствует принципу однонаправленного потока данных Vue.

Панель содержит три элемента управления. Поле текстового поиска с иконкой лупы фильтрует записи по вхождению строки в название или описание фильма. Выпадающий список жанров формируется динамически из входного параметра `categories`, содержащего массив уникальных жанров, присутствующих в текущем списке фильмов. Выпадающий список статусов позволяет отобразить только просмотренные или только запланированные к просмотру фильмы. При наличии любого активного фильтра появляется кнопка сброса, сбрасывающая все три поля в пустые значения одновременно.

2.6 Реализация страниц приложения

Страницы приложения расположены в папке `views` и представляют собой компоненты, напрямую связанные с маршрутами роутера. Каждая страница компонуется компоненты из папки `components`, получает данные и методы из `composables` и реализует логику, специфичную для конкретного раздела приложения. В данном разделе рассматриваются все девять страниц приложения `CineTrack`.

2.6.1. Страница авторизации (LoginView)

Страница авторизации является единственным публично доступным маршрутом приложения и отображается всем пользователям, не имеющим активной сессии. Страница реализует два режима работы в рамках одного компонента: вход для существующих пользователей и регистрацию новых.

Переключение между режимами осуществляется через локальную реактивную переменную `mode`, принимающую значения `login` или `register`. При смене режима сбрасываются все ошибки валидации и глобальное сообщение об ошибке, поля формы при этом не очищаются, что удобно в ситуации, когда пользователь ввёл данные и решил переключиться с регистрации на вход.

При отправке формы вызывается метод `validate`, проверяющий длину логина и пароля. При успешной валидации вызывается соответствующий метод `composable useAuth` — `login` или `register`. Результат содержит поле `ok` булевого типа: при значении `true` роутер выполняет переход на главную страницу, при значении `false` текст ошибки из поля `error` отображается в блоке над кнопкой отправки.

2.6.2. Главная страница (HomeView)

Главная страница является центральным разделом приложения и реализует наибольший объём интерактивной функциональности. Страница получает данные и методы из двух `composables`: `useItems` предоставляет список фильмов и счётчики, `useSettings` предоставляет флаг `confirmDelete`.

В верхней части страницы располагается блок заголовка с названием раздела и кнопкой перехода на страницу создания фильма. Под заголовком выводится строка счётчиков, отображающая три значения: общее количество активных записей, количество запланированных к просмотру и количество просмотренных. Счётчики реализованы как

вычисляемые свойства `composable useItems` и обновляются автоматически при любом изменении списка.

Фильтрация реализована через локальный реактивный объект `filters` с тремя полями, передаваемый в компонент `ItemFilters` через директиву `v-model`. Отфильтрованный список вычисляется в локальном вычисляемом свойстве `filteredItems`, последовательно применяющем три фильтра: текстовый поиск по названию и описанию без учёта регистра, фильтр по жанру и фильтр по статусу просмотра. Поскольку `filteredItems` является вычисляемым свойством, список автоматически пересчитывается при изменении любого фильтра или при изменении массива `items` в `composable`.

Список фильмов передаётся в компонент `ItemList`, который обрабатывает события `toggle` и `delete`. Обработчик `handleToggle` вызывает метод `toggleDone` `composable`. Обработчик `handleDelete` проверяет флаг `confirmDelete`: если флаг активен, перед удалением отображается стандартный диалог браузера `confirm` с запросом подтверждения; при отказе пользователя операция отменяется. При подтверждении или при неактивном флаге вызывается метод `softDelete`, перемещающий запись в архив.

При отсутствии записей, соответствующих текущим фильтрам, отображается блок пустого состояния. Его содержимое зависит от контекста: если массив `activeItems` пуст полностью, предлагается добавить первый фильм со ссылкой на страницу создания; если записи существуют, но ни одна не соответствует фильтрам, выводится соответствующее сообщение.

2.6.3. Страница создания фильма (CreateView)

Страница создания содержит заголовок с навигационной ссылкой возврата на главную страницу и компонент `ItemForm`. Входной параметр `initial` не передаётся, поэтому форма использует пустые значения по умолчанию. Параметр `submitLabel` установлен в значение «Добавить в список».

При генерации компонентом `ItemForm` события `submit` страница вызывает метод `addItem` `composable useItems`, передавая полученный объект данных. После успешного добавления роутер выполняет программный переход на главную страницу через вызов `router.push('/')`, где новая запись сразу отображается в списке благодаря реактивности `singleton composable`.

2.6.4. Страница редактирования фильма (EditView)

Страница редактирования получает идентификатор фильма из параметра маршрута `route.params.id` и с его помощью находит запись через метод `getItemById` `composable useItems`. Поиск выполняется в вычисляемом свойстве, что обеспечивает реактивное обновление при изменении параметра маршрута без перезагрузки компонента.

Страница реализует два сценария отображения. Если запись с указанным идентификатором не найдена, выводится блок с сообщением об ошибке, содержащий идентификатор из URL и кнопку возврата на главную страницу. Это обрабатывает ситуацию ручного ввода несуществующего URL. Если запись найдена, отображается компонент `ItemForm` с предзаполненными данными: найденный объект передаётся в параметр `initial`, а `submitLabel` установлен в значение «Сохранить изменения».

При генерации события `submit` вызывается метод `updateItem` `composable useItems` с идентификатором записи и обновлёнными данными. После сохранения роутер выполняет переход на главную страницу.

2.6.5. Страница архива (ArchiveView)

Страница архива отображает список мягко удалённых записей, полученных из вычисляемого свойства `archivedItems` `composable useItems`. В заголовке страницы рядом с названием отображается счётчик архивных записей, стилизованный в красный цвет для привлечения внимания.

При пустом архиве отображается блок пустого состояния с соответствующим сообщением. При наличии архивных записей каждая из них отображается в виде строки, содержащей жанровую метку, зачёркнутое название, дату удаления и две кнопки действий.

Кнопка «Восстановить» вызывает метод `restoreItem`, устанавливающий поле `deletedAt` записи в значение `null`. Запись немедленно исчезает из `archivedItems` и появляется в `activeItems` благодаря реактивности вычисляемых свойств. Кнопка «Удалить навсегда» вызывает метод `deleteForever` после необязательного подтверждения через диалог браузера. Эта операция необратима: запись физически удаляется из массива и из `LocalStorage`.

2.6.6. Страница статистики (StatsView)

Страница статистики предоставляет пользователю агрегированный обзор его списка фильмов. Страница получает данные исключительно из `composable useItems` и не содержит собственного изменяемого состояния.

В верхней части страницы располагается карточка общего прогресса, содержащая три элемента: строку с подписью и дробным значением просмотренных к общему количеству фильмов, горизонтальный прогресс-бар и текстовую подпись с процентным значением. Ширина заполненной части прогресс-бара вычисляется как отношение `doneCount` к `totalCount`, умноженное на сто, и устанавливается через `inline-стиль`. Изменение ширины сопровождается CSS-переходом длительностью 0,5 секунды, что создаёт плавную анимацию при изменении данных.

Ниже располагается сетка из четырёх карточек-счётчиков, отображающих общее количество фильмов в списке, количество запланированных к просмотру, количество просмотренных и количество записей в архиве. Каждая карточка стилизована собственным акцентным цветом рамки и числового значения.

В нижней части страницы отображается разбивка по жанрам в виде списка горизонтальных полос. Данные для разбивки вычисляются в локальном вычисляемом свойстве `byGenre`: метод перебирает массив `activeItems`, подсчитывает количество записей каждого жанра, формирует массив объектов с полями `name` и `count` и сортирует его по убыванию количества. Ширина каждой полосы вычисляется как отношение количества фильмов данного жанра к общему количеству активных фильмов.

2.6.7. Страница настроек (SettingsView)

Страница настроек предоставляет пользователю интерфейс управления параметрами приложения. Страница получает текущие значения настроек и методы их изменения из `composable useSettings` и не содержит собственного локального состояния.

Страница разделена на четыре секции. Секция выбора темы содержит две кнопки-переключателя — «Светлая» и «Тёмная» — с визуальным выделением активного значения. Нажатие вызывает метод `setTheme`, который обновляет переменную `theme`, применяет атрибут `data-theme` к корневому элементу документа и сохраняет настройку в `LocalStorage`.

Секция выбора цветовой схемы содержит четыре кнопки с цветными точками, соответствующими каждой схеме. Нажатие вызывает метод `setColorScheme`, аналогично применяющий атрибут `data-scheme` и сохраняющий выбор.

Секция режима отображения содержит две кнопки-переключателя: «Карточки» и «Таблица». Выбранный режим немедленно применяется на главной странице, поскольку компонент `ItemList` читает `viewMode` из того же `singleton composable`.

Секция безопасности содержит переключатель в виде кастомного элемента `toggle`, управляющего флагом `confirmDelete`. При активном значении перед каждым удалением на главной странице и в архиве отображается диалог подтверждения.

2.6.8. Страница о проекте (AboutView) и страница 404 (NotFoundView)

Страница `AboutView` содержит статический текст с описанием проекта, использованного технологического стека и реализованного функционала. Страница не использует `composables` и не содержит интерактивных элементов.

Страница `NotFoundView` отображается при обращении к любому несуществующему маршруту благодаря записи с шаблоном `/:pathMatch(.)` в конфигурации роутера. Страница содержит крупный декоративный текст «404», заголовок с сообщением об ошибке и кнопку возврата на главную страницу. Визуально число 404 выводится с низкой непрозрачностью и акцентным цветом, создавая декоративный фоновый элемент.

2.7 Тестирование приложения на соответствие требованиям

Тестирование является завершающим этапом практической разработки и позволяет убедиться в том, что реализованное приложение соответствует всем функциональным и техническим требованиям, сформулированным в первой главе. В данном разделе приводятся результаты проверки ключевых сценариев работы приложения.

Тестирование маршрутизации и навигационного охранника

Первым шагом проверяется корректность работы маршрутизации и защиты маршрутов. При открытии приложения без активной сессии выполняется попытка прямого перехода по адресу главной страницы. Навигационный охранник `beforeEach` перехватывает переход, обнаруживает отсутствие сессии в `LocalStorage` и выполняет автоматическое перенаправление на страницу авторизации. Пользователь оказывается на странице `/login`, что подтверждает корректную работу защиты маршрутов.

После успешной регистрации выполняется попытка ручного перехода по адресу `/login`. Охранник обнаруживает активную сессию и перенаправляет пользователя на главную страницу, исключая повторное отображение формы авторизации для авторизованного пользователя.

Проверяются все восемь маршрутов приложения путём последовательного перехода по каждому из них. Все переходы выполняются без перезагрузки страницы, что подтверждает корректную работу клиентской маршрутизации. Индикатор загрузки появляется в начале каждого перехода и исчезает через 400 миллисекунд после его завершения. При переходе по несуществующему адресу, например `/nonexistent`, отображается страница 404 с кнопкой возврата на главную страницу.

Тестирование авторизации и регистрации

Проверяется сценарий регистрации нового пользователя. Вводится логин из трёх и более символов и пароль из четырёх и более символов. После нажатия кнопки «Создать аккаунт» в LocalStorage под ключом cinetrack_users появляется массив с одним объектом пользователя, содержащим хэшированный пароль. Под ключом cinetrack_session появляется объект сессии. Приложение выполняет переход на главную страницу, в шапке отображается логин зарегистрированного пользователя.

Проверяется защита от дублирования логина. При попытке повторной регистрации с тем же логином форма отображает сообщение об ошибке «Пользователь с таким логином уже существует» без перехода на другую страницу.

Проверяется сценарий входа существующего пользователя. После выхода из системы вводятся корректные логин и пароль — приложение выполняет переход на главную страницу. При вводе неверного пароля отображается сообщение «Неверный логин или пароль».

Проверяется валидация формы. При попытке отправить форму с пустыми полями под каждым полем появляются сообщения об ошибках, отправка не выполняется. При вводе логина из двух символов появляется сообщение о минимальной длине.

Тестирование операций с данными

Проверяется полный цикл операций с записями фильмов. Через страницу создания добавляется новый фильм с заполнением всех трёх полей. После нажатия кнопки «Добавить в список» выполняется переход на главную страницу, где новая запись немедленно отображается в списке. В инструментах разработчика браузера в разделе LocalStorage под ключом cinetrack_items обнаруживается обновлённый массив с добавленной записью. Это подтверждает корректную синхронизацию состояния приложения с LocalStorage после каждого изменения.

Проверяется редактирование существующей записи. При нажатии кнопки редактирования на карточке фильма выполняется переход на страницу /edit/:id, форма предзаполнена актуальными данными выбранного фильма. После изменения названия и сохранения главная страница отображает обновлённое название. Содержимое LocalStorage также обновляется.

Проверяется функция отметки просмотра. При нажатии кнопки отметки на карточке фильма его статус изменяется: карточка приобретает зеленоватый оттенок, заголовок отображается с зачёркиванием, в правом верхнем углу карточки появляется метка «Просмотрено». Счётчики в верхней части главной страницы немедленно обновляются. Повторное нажатие снимает отметку и возвращает карточку к исходному виду.

Проверяется мягкое удаление и восстановление. При нажатии кнопки удаления на карточке фильма при активном флаге confirmDelete появляется стандартный диалог браузера с запросом подтверждения. После подтверждения запись исчезает из главной страницы. На странице архива запись появляется в списке с зачёркнутым названием и датой удаления. После нажатия кнопки «Восстановить» запись исчезает из архива и снова появляется на главной странице. После нажатия кнопки «Удалить навсегда» запись исчезает из архива и не появляется нигде в приложении.

Тестирование сохранения данных после перезагрузки

Выполняется проверка сохранения состояния после перезагрузки страницы, являющаяся ключевым требованием к реализации LocalStorage. В список добавляется несколько фильмов, один отмечается просмотренным, один удаляется в архив. Затем

страница браузера перезагружается через клавишу F5. После перезагрузки приложение восстанавливает все данные: список фильмов отображается в том же составе, статусы просмотра сохранены, архивные записи присутствуют на странице архива. Настройки темы и режима отображения также восстанавливаются. Сессия пользователя сохраняется, страница авторизации не отображается.

Тестирование фильтрации

Проверяется корректность работы всех трёх фильтров. При вводе части названия фильма в поле поиска список немедленно обновляется, отображая только совпадающие записи. При выборе жанра из выпадающего списка отображаются только фильмы выбранного жанра. При выборе статуса «Просмотрено» отображаются только записи с активным флагом `isDone`. Комбинированное применение нескольких фильтров одновременно работает корректно: отображаются только записи, удовлетворяющие всем активным условиям одновременно. Кнопка сброса появляется при наличии любого активного фильтра и сбрасывает все три поля одновременно.

Тестирование адаптивности

Проверяется корректность отображения приложения на экранах различной ширины с использованием инструментов разработчика браузера в режиме эмуляции устройств.

При ширине экрана 1280 пикселей отображается полная десктопная версия интерфейса: горизонтальное навигационное меню, блок пользователя с логином и кнопкой выхода, список фильмов в виде сетки из трёх колонок.

При ширине экрана 768 пикселей горизонтальное меню скрывается, вместо него отображается кнопка-бургер. Сетка карточек перестраивается в две колонки. Элементы форм занимают всю доступную ширину.

При ширине экрана 375 пикселей, соответствующей типичному смартфону, мобильное меню раскрывается вертикально при нажатии кнопки-бургер. Карточки отображаются в одну колонку. Кнопки форм перестраиваются вертикально. Все текстовые элементы остаются читаемыми без горизонтальной прокрутки страницы.

При ширине экрана 320 пикселей, соответствующей минимальному поддерживаемому размеру, все элементы интерфейса остаются функциональными и читаемыми.

Проверка изоляции сервисного слоя

Выполняется проверка соблюдения требования об отсутствии прямых обращений к `LocalStorage` вне сервисного слоя. Поиск строки `localStorage` в файлах папок `components`, `views` и `composables` не обнаруживает ни одного вхождения. Все обращения к `LocalStorage` сосредоточены исключительно в файлах `services/storage.js` и `services/auth.js`, что подтверждает соблюдение архитектурного требования об изоляции сервисного слоя.

Результаты тестирования

По итогам проведённого тестирования все функциональные и технические требования, сформулированные в разделе 1.1, признаны выполненными. Приложение корректно работает в актуальных версиях браузеров Google Chrome, Mozilla Firefox и Microsoft Edge. Данные сохраняются между сессиями браузера. Интерфейс корректно адаптируется к экранам от 320 пикселей. Архитектурные требования к изоляции сервисного слоя, реализации `singleton composables` и использованию навигационных охранников соблюдены в полном объёме.

Проверяется работа поиска по глобальной библиотеке TMDb. При вводе названия фильма в поле поиска на странице создания запрос к API выполняется с задержкой 350 миллисекунд после последнего введенного символа. Выпадающий список отображает до шести результатов с постерами, жанрами и рейтингами. При выборе фильма поля названия, описания и жанра заполняются автоматически, постер сохраняется вместе с записью. Проверяется переход на страницу отдельного фильма по нажатию на карточку — страница отображает полное описание, постер, метаданные и кнопки управления записью.

ЗАКЛЮЧЕНИЕ

В ходе выполнения данной курсовой работы было спроектировано и реализовано одностраничное web-приложение CineTrack, предназначенное для персонального учёта фильмов. Приложение обеспечивает полный цикл работы с данными, включая добавление, редактирование, мягкое удаление с возможностью восстановления, фильтрацию по нескольким параметрам, просмотр статистики и настройку внешнего вида интерфейса.

В процессе работы были решены все задачи, поставленные во введении.

Проведён анализ предметной области, в ходе которого были изучены существующие аналогичные решения — Letterboxd, Кинопоиск и IMDb Lists. Анализ показал, что существующие сервисы ориентированы на широкую аудиторию и перегружены функционалом, тогда как разработанное приложение занимает нишу лёгкого автономного трекера, не требующего серверной инфраструктуры и хранящего данные локально в браузере пользователя.

Спроектирована структура страниц и навигационная система приложения, включающая девять маршрутов с разграничением публичного и защищённого доступа. Разработаны wireframe-макеты ключевых экранов и обоснована визуальная концепция интерфейса, основанная на кинематографической цветовой палитре и сочетании двух типографических семейств.

Обоснован выбор технологического стека. Применение Vue 3 с Composition API и синтаксисом script setup обеспечило компонентную архитектуру с чистым и лаконичным кодом. Использование Vite как инструмента сборки обеспечило высокую скорость разработки. Vue Router с навигационными охранниками реализовал как клиентскую маршрутизацию, так и защиту маршрутов от неавторизованного доступа. Изоляция работы с LocalStorage в отдельном сервисном слое обеспечила централизованную обработку ошибок и единое хранение ключей хранилища.

Реализована компонентная архитектура приложения, включающая семь переиспользуемых компонентов и девять компонентов-страниц. Все composables реализованы по принципу singleton, обеспечивающему единое согласованное состояние без применения внешних библиотек управления состоянием.

Реализован сервисный слой, полностью изолирующий остальную кодовую базу от прямого взаимодействия с LocalStorage. Сервисы storage.js и auth.js обеспечивают надёжное чтение и запись данных с обработкой исключений через блоки try/catch.

Реализована система авторизации на основе LocalStorage, включающая регистрацию новых пользователей, аутентификацию по логину и хэшированному паролю, управление сессией и защиту всех маршрутов приложения посредством навигационного охранника роутера.

Проведено тестирование готового приложения по всем ключевым сценариям: маршрутизация и защита маршрутов, авторизация и регистрация, полный цикл операций с данными, сохранение состояния после перезагрузки страницы, фильтрация, адаптивность и изоляция сервисного слоя. Все проверки подтвердили соответствие приложения техническим требованиям.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. You E. Vue 3. Официальная документация [Электронный ресурс] / E. You. — 2024. — Режим доступа: <https://vuejs.org/guide/introduction>. — Дата обращения: 20.04.2026.
2. Vue Router 4. Официальная документация [Электронный ресурс]. — 2024. — Режим доступа: <https://router.vuejs.org/guide>. — Дата обращения: 20.04.2026.
3. Vite. Официальная документация [Электронный ресурс]. — 2024. — Режим доступа: <https://vitejs.dev/guide>. — Дата обращения: 20.04.2026.
4. Web Storage API [Электронный ресурс] // MDN Web Docs / Mozilla Foundation. — 2024. — Режим доступа: https://developer.mozilla.org/en-US/docs/Web/API/Web_Storage_API. — Дата обращения: 21.04.2026.
5. CSS Custom Properties (переменные) [Электронный ресурс] // MDN Web Docs / Mozilla Foundation. — 2024. — Режим доступа: https://developer.mozilla.org/en-US/docs/Web/CSS/Using_CSS_custom_properties. — Дата обращения: 21.04.2026.
6. Crypto: randomUUID() [Электронный ресурс] // MDN Web Docs / Mozilla Foundation. — 2024. — Режим доступа: <https://developer.mozilla.org/en-US/docs/Web/API/Crypto/randomUUID>. — Дата обращения: 21.04.2026.
7. Флэнаган, Д. JavaScript: подробное руководство / Д. Флэнаган. — 7-е изд. — СПб. : БХВ-Петербург, 2021. — 704 с. — ISBN 978-5-9775-6741-9.
8. Симпсон, К. Вы не знаете JS: ES6 и не только / К. Симпсон. — СПб. : Питер, 2017. — 333 с. — ISBN 978-5-496-02473-6.
9. Marquardt, J. Vue.js 3 Design Patterns and Best Practices / J. Marquardt. — Birmingham : Packt Publishing, 2023. — 296 p. — ISBN 978-1-80324-845-1.
10. Syne. Typeface specimen [Электронный ресурс] // Google Fonts. — Режим доступа: <https://fonts.google.com/specimen/Syne>. — Дата обращения: 22.04.2026.
11. DM Sans. Typeface specimen [Электронный ресурс] // Google Fonts. — Режим доступа: <https://fonts.google.com/specimen/DM+Sans>. — Дата обращения: 22.04.2026.
12. Letterboxd — социальная сеть для любителей кино [Электронный ресурс]. — Режим доступа: <https://letterboxd.com>. — Дата обращения: 22.04.2026.
13. Кинопоиск [Электронный ресурс] / ООО «Яндекс». — Режим доступа: <https://www.kinopoisk.ru>. — Дата обращения: 22.04.2026.
14. IMDb — Internet Movie Database [Электронный ресурс] / Amazon.com, Inc. — Режим доступа: <https://www.imdb.com>. — Дата обращения: 22.04.2026.

Приложение 1

Секция с настройками

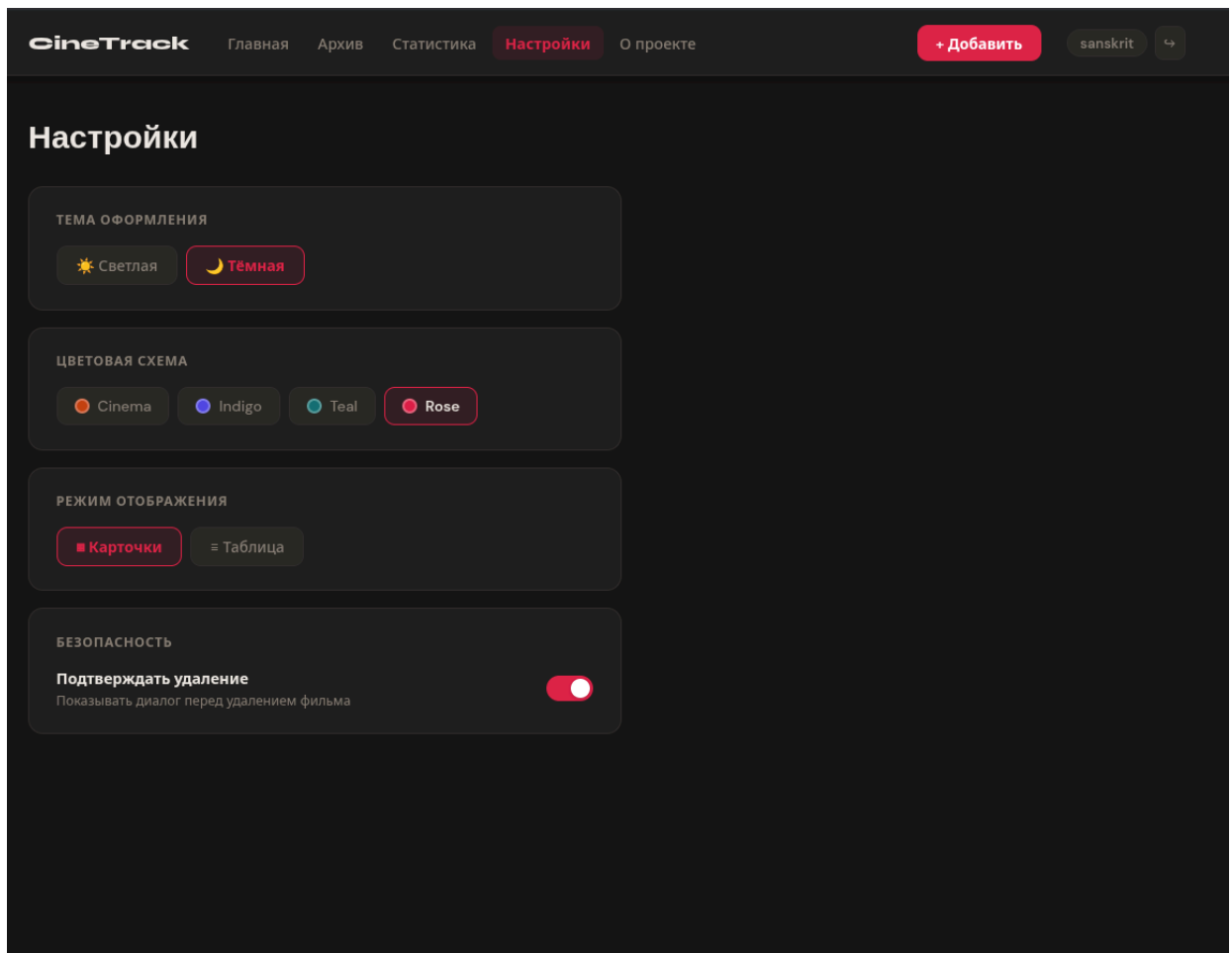


Рисунок 1 - Секция с настройками

Примечание - составлено автором на основе проведенного исследования


```
// Guard: показать лоадер + проверить авторизацию
router.beforeEach((to) => {
  showLoader()

  // Если маршрут не публичный и нет сессии - редирект на /login
  const isLoggedIn = !!getSession()
  if (!to.meta.public && !isLoggedIn) {
    return '/login'
  }

  // Если уже залогинен и пытается зайти на /login - на главную
  if (to.path === '/login' && isLoggedIn) {
    return '/'
  }
})
```

Рисунок 2 — Охранник beforeEach (router/index.js)

Примечание - составлено автором на основе проведенного исследования